

Élasticité linéaire 2D

Vincent Degrooff

April 22, 2025

Contents

1	Modèle constitutif	2
1.1	Déformations planes	3
1.2	Tensions planes	3
1.3	Axisymétrie	4
2	Éléments finis	5
2.1	Formulation faible	5
2.2	Formulation discrète	5
2.3	Matrices locales	6
2.4	Conditions naturelles de Neumann et Robin	8
2.5	Conditions essentielles de Dirichlet	9
2.6	Calcul des contraintes	11
2.7	Analyse modale	12
3	Résolution numérique	14
3.1	Système linéaire	14
3.2	Mode propre	16
3.3	Adimensionalisation	18
4	Validation	20
4.1	Poutre encastree	20
4.2	Cylindre creux	22
4.3	Solutions numériques	23

1 Modèle constitutif

On s'intéresse à la mécanique du solide dans l'hypothèse de petites déformations. Les particules matérielles d'un corps soumis à différentes forces de volume et de surface se déplacent d'une position $\mathbf{X}$ à une position $\mathbf{x}(\mathbf{X}, t)$. Le déplacement se note $\mathbf{u} = \mathbf{x} - \mathbf{X}$ et satisfait les équations suivantes :

$$\rho \partial_t^2 \mathbf{u} = \nabla \cdot \boldsymbol{\sigma} + \mathbf{f} \quad \text{Conservation de la quantité de mouvement} \quad (1)$$

$$\boldsymbol{\epsilon} = \frac{1}{2} (\nabla \mathbf{u} + \nabla^T \mathbf{u}) \quad \text{Tenseur des déformations infinitésimales} \quad (2)$$

$$\boldsymbol{\sigma} = \mathbf{C} : \boldsymbol{\epsilon} \quad \text{Équation constitutive.} \quad (3)$$

Les tenseurs symétriques de déformation $\boldsymbol{\epsilon}$ et de contraintes $\boldsymbol{\sigma}$, sont liés par une loi constitutive à travers le tenseur des rigidités $\mathbf{C}$ d'ordre 4. Dans le cas d'un matériau isotrope, on trouve la relation

$$\boldsymbol{\sigma} = 2\mu\boldsymbol{\epsilon} + \lambda \text{tr}(\boldsymbol{\epsilon})\mathbf{I} \quad (4)$$

$$\mu = \frac{E}{2(1 + \nu)} \quad (5)$$

$$\lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)} \quad (6)$$

où

- E [Pa], le module de Young, relie la déformation à la contrainte de traction / compression.
- ν [-], le coefficient de Poisson, quantifie la déformation d'un matériau dans la direction perpendiculaire à l'effort.

Bien que la physique du solide soit évidemment tri-dimensionnelle, il est souvent possible de réduire le problème à deux dimensions, voire une seule. Ici, nous allons nous intéresser à des cas particuliers où l'on peut négliger l'influence de la troisième dimension :

- **Déformations planes** : la section d'un solide de grande profondeur ne peut pas se déformer dans cette direction.
- **Tension planes** : un solide de faible épaisseur ne subit pas de déformation dans la direction de l'épaisseur.
- **Axisymétrie** : un solide de révolution soumis à des forces axisymétriques se déforme indépendamment de la position angulaire.

1.1 Déformations planes

En considérant un déplacement uniquement planaire $\mathbf{u} = u(x, y)\mathbf{e}_1 + v(x, y)\mathbf{e}_2$, dont on peut négliger la dépendance en z , on obtient les tenseurs de déformation, et de contrainte grâce à l'équation constitutive (4) :

$$\boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_{xx} & \epsilon_{xy} & 0 \\ \epsilon_{xy} & \epsilon_{yy} & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} u_x & (u_x + v_y)/2 & 0 \\ (u_x + v_y)/2 & v_y & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (7)$$

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & 0 \\ \sigma_{xy} & \sigma_{yy} & 0 \\ 0 & 0 & \sigma_{zz} \end{bmatrix} \quad (8)$$

$$\begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \end{bmatrix} = \underbrace{\begin{bmatrix} \gamma & \lambda & \lambda \\ \lambda & \gamma & \lambda \\ \lambda & \lambda & \gamma \\ & & \mu \end{bmatrix}}_{\mathbf{C}} \begin{bmatrix} \epsilon_{xx} \\ \epsilon_{yy} \\ 0 \\ 2\epsilon_{xy} \end{bmatrix} \quad (9)$$

$$\text{où } \gamma \triangleq 2\mu + \lambda = \frac{E(1-\nu)}{(1+\nu)(1-2\nu)}. \quad (10)$$

1.2 Tensions planes

En considérant que la pièce est soumise à des contraintes dans le plan xy uniquement, on force $0 = \sigma_{zz}$:

$$\begin{aligned} \sigma_{zz} &= \lambda\epsilon_{xx} + \lambda\epsilon_{yy} + (2\mu + \lambda)\epsilon_{zz} = 0 \\ \iff \epsilon_{zz} &= -\frac{\nu}{1-\nu}(\epsilon_{xx} + \epsilon_{yy}) \end{aligned} \quad (11)$$

En considérant à nouveau un déplacement $\mathbf{u} = u(x, y)\mathbf{e}_1 + v(x, y)\mathbf{e}_2$, on obtient

$$\boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_{xx} & \epsilon_{xy} & 0 \\ \epsilon_{xy} & \epsilon_{yy} & 0 \\ 0 & 0 & \epsilon_{zz} \end{bmatrix} = \begin{bmatrix} u_x & (u_x + v_y)/2 & 0 \\ (u_x + v_y)/2 & v_y & 0 \\ 0 & 0 & -\frac{\nu}{1-\nu}(u_x + v_y) \end{bmatrix} \quad (12)$$

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & 0 \\ \sigma_{xy} & \sigma_{yy} & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (13)$$

$$\begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{xy} \end{bmatrix} = \underbrace{\begin{bmatrix} \alpha & \beta \\ \beta & \alpha \\ & \mu \end{bmatrix}}_{\mathbf{C}} \begin{bmatrix} \epsilon_{xx} \\ \epsilon_{yy} \\ 2\epsilon_{xy} \end{bmatrix} \quad \begin{aligned} \alpha &= \frac{E}{1-\nu^2} \\ \beta &= \frac{E\nu}{1-\nu^2} \end{aligned} \quad (14)$$

1.3 Axisymétrie

En considérant un déplacement axisymétrique $\mathbf{u} = u(r, z)\mathbf{e}_r + v(r, z)\mathbf{e}_\theta + w(r, z)\mathbf{e}_z$, et en utilisant le gradient en coordonnées cylindriques, on obtient les relations

$$\boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_{rr} & \epsilon_{r\theta} & \epsilon_{rz} \\ 0 & \epsilon_{\theta\theta} & 0 \\ \epsilon_{rz} & 0 & \epsilon_{zz} \end{bmatrix} = \begin{bmatrix} u_r & (-v/r + v_r)/2 & (u_z + w_r)/2 \\ (-v/r + v_r)/2 & u/r & v_z/2 \\ (u_z + w_r)/2 & v_z/2 & w_z \end{bmatrix} \quad (15)$$

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{rr} & \sigma_{r\theta} & \sigma_{rz} \\ \sigma_{r\theta} & \sigma_{\theta\theta} & \sigma_{\theta z} \\ \sigma_{rz} & \sigma_{\theta z} & \sigma_{zz} \end{bmatrix} \quad (16)$$

$$\begin{bmatrix} \sigma_{rr} \\ \sigma_{\theta\theta} \\ \sigma_{zz} \\ \sigma_{rz} \\ \sigma_{r\theta} \\ \sigma_{\theta z} \end{bmatrix} = \underbrace{\begin{bmatrix} \gamma & \lambda & \lambda & & & \\ \lambda & \gamma & \lambda & & & \\ \lambda & \lambda & \gamma & & & \\ & & & \mu & & \\ & & & & \mu & \\ & & & & & \mu \end{bmatrix}}_{\mathbf{C}} \begin{bmatrix} \epsilon_{rr} \\ \epsilon_{\theta\theta} \\ \epsilon_{zz} \\ 2\epsilon_{rz} \\ 2\epsilon_{r\theta} \\ 2\epsilon_{\theta z} \end{bmatrix}. \quad (17)$$

Cependant, la composante tangentielle v du déplacement est supposée nulle dans le code, ce qui nous permet de réduire la matrice $\mathbf{C}$ à ses 4 premières lignes et colonnes :

$$\begin{bmatrix} \sigma_{rr} \\ \sigma_{\theta\theta} \\ \sigma_{zz} \\ \sigma_{rz} \end{bmatrix} = \underbrace{\begin{bmatrix} \gamma & \lambda & \lambda & \\ \lambda & \gamma & \lambda & \\ \lambda & \lambda & \gamma & \\ & & & \mu \end{bmatrix}}_{\mathbf{C}} \begin{bmatrix} \epsilon_{rr} \\ \epsilon_{\theta\theta} \\ \epsilon_{zz} \\ 2\epsilon_{rz} \end{bmatrix}. \quad (18)$$

2 Éléments finis

2.1 Formulation faible

Soit un corps $\Omega \subseteq \mathbb{R}^d$ avec une frontière $\Gamma = \partial\Omega$ partitionnée en

- Γ_D : déplacement imposé (Dirichlet),
- Γ_N : contrainte imposée (Neumann),
- Γ_R : relation contrainte-déplacement imposée (Robin).

Les contraintes peuvent être imposées sur le vecteur complet, qu'il s'agisse du déplacement ou de la contrainte, ou bien seulement sur une de ses composantes x , y voire même selon une direction normale, tangente, ou arbitraire. Notons $\mathcal{M}(\mathbf{x})$ l'ensemble des déplacements $\mathbf{u}(\mathbf{x})$ admissibles sur un point de Γ_D . Dans le cas d'un déplacement normal, on a par exemple

$$\mathcal{M}(\mathbf{x}) \triangleq \{ \mathbf{u}(\mathbf{x}) \in \mathbb{R}^d \quad \text{t.q.} \quad \mathbf{u}(\mathbf{x}) \cdot \mathbf{n}(\mathbf{x}) = 0 \}. \quad (19)$$

En suivant l'approche de Galerkin, nous cherchons une solution $\mathbf{u} \in \mathcal{U}(\Omega)$, l'espace des solutions admissibles, telle que la formulation faible soit vérifiée $\forall \mathbf{v} \in \mathcal{V}(\Omega)$, l'espace des fonctions test :

$$\mathcal{U} \triangleq \{ \mathbf{u} \in [H^1(\Omega)]^d \quad \text{t.q.} \quad \mathbf{u}(\mathbf{x}) - \mathbf{u}_D(\mathbf{x}) \in \mathcal{M}(\mathbf{x}) \quad \forall \mathbf{x} \in \Gamma_D \}, \quad (20)$$

$$\mathcal{V} \triangleq \{ \mathbf{v} \in [H^1(\Omega)]^d \quad \text{t.q.} \quad \mathbf{v}(\mathbf{x}) \in \mathcal{M}(\mathbf{x}) \quad \forall \mathbf{x} \in \Gamma_D \}. \quad (21)$$

En partant de l'équation de conservation de la quantité de mouvement, on obtient :

$$\begin{aligned} \int_{\Omega} \rho \partial_t^2 \mathbf{u} \cdot \mathbf{v} \, d\Omega &= \int_{\Omega} (\nabla \cdot \boldsymbol{\sigma}) \cdot \mathbf{v} \, d\Omega + \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega \\ &= \int_{\Omega} [\nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{v}) - \boldsymbol{\sigma} : \nabla \mathbf{v}] \, d\Omega + \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega \\ &= - \int_{\Omega} \boldsymbol{\sigma} : [\boldsymbol{\epsilon}(\mathbf{v}) + \boldsymbol{\omega}(\mathbf{v})] \, d\Omega + \int_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma} \cdot \mathbf{v} \, d\Gamma + \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega \\ &= - \int_{\Omega} \boldsymbol{\sigma}(\mathbf{u}) : \boldsymbol{\epsilon}(\mathbf{v}) \, d\Omega + \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega + \int_{\Gamma_N \cup \Gamma_R} \mathbf{t} \cdot \mathbf{v} \, d\Gamma \quad \forall \mathbf{v} \in \mathcal{V}(\Omega). \end{aligned} \quad (22)$$

2.2 Formulation discrète

On fait maintenant l'hypothèse que la solution $\mathbf{u}$ peut être approchée par une approximation linéaire par morceaux $\mathbf{u}^h$. Chaque morceau est un élément (triangle / quadrilatère) d'un maillage $\mathcal{T}_h$ de Ω , avec n nœuds et N éléments. On note $\phi_j(\mathbf{x})$ la fonction de forme scalaire associée au nœud j et $\mathbf{u}_j$ la valeur du déplacement au nœud j . On peut donc écrire la solution approchée

$$\mathbf{u}^h(\mathbf{x}) = \sum_{j=1}^n \phi_j(\mathbf{x}) \mathbf{u}_j. \quad (23)$$

Étant donné qu'il y a $2n$ degrés de liberté, la base de l'espace discret $\mathcal{U}^h$ contient $2n$ éléments :

$$\begin{aligned}\mathcal{U}^h &= \left\{ \begin{bmatrix} \phi_1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \phi_1 \end{bmatrix}, \begin{bmatrix} \phi_2 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \phi_2 \end{bmatrix}, \dots, \begin{bmatrix} \phi_n \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \phi_n \end{bmatrix} \right\} \\ &= \left\{ \boldsymbol{\varphi}_1, \boldsymbol{\psi}_1, \boldsymbol{\varphi}_2, \boldsymbol{\psi}_2, \dots, \boldsymbol{\varphi}_n, \boldsymbol{\psi}_n \right\} \\ &= \left\{ \boldsymbol{\tau}_1, \boldsymbol{\tau}_2, \boldsymbol{\tau}_3, \boldsymbol{\tau}_4, \dots, \boldsymbol{\tau}_{2n-1}, \boldsymbol{\tau}_{2n} \right\}\end{aligned}\tag{24}$$

On peut ainsi réécrire l'approximation de la solution $\mathbf{u}^h$ comme

$$\mathbf{u}^h(\mathbf{x}) = \sum_{j=1}^n \boldsymbol{\varphi}_j(\mathbf{x}) u_j + \boldsymbol{\psi}_j(\mathbf{x}) v_j = \sum_{j=1}^{2n} \boldsymbol{\tau}_j(\mathbf{x}) q_j \tag{25}$$

En remplaçant $\mathbf{u}$ par $\mathbf{u}^h$ dans (22), on obtient la formulation discrète :

$$\begin{aligned}& \sum_{j=1}^{2n} \left[\int_{\Omega} \rho \boldsymbol{\tau}_i \cdot \boldsymbol{\tau}_j \, d\Omega \right] \partial_t^2 q_j \\ & + \sum_{j=1}^{2n} \left[\int_{\Omega} \boldsymbol{\epsilon}(\boldsymbol{\tau}_i) \cdot \mathbf{C} \cdot \boldsymbol{\epsilon}(\boldsymbol{\tau}_j) \, d\Omega \right] q_j \\ & = \int_{\Omega} \boldsymbol{\tau}_i \cdot \mathbf{f} \, d\Omega + \int_{\Gamma_N \cup \Gamma_R} \boldsymbol{\tau}_i \cdot \mathbf{t} \, d\Gamma\end{aligned}\tag{26}$$

que l'on peut réécrire sous la forme matricielle

$$M\ddot{\mathbf{q}} + K\mathbf{q} = \mathbf{b}.\tag{27}$$

Dans le cas plane, toutes les intégrales de volume présentées ci-dessus sont calculées dans un volume tri-dimensionnel : $\int_{\Omega} = \int \int \int dx dy dz$. Cependant, aucune intégrande ne dépend de la coordonnée z et on peut donc omettre une des intégrales des deux côtés de l'équation.

Dans le cas axisymétrique, les intégrales de volume sont également calculées dans un volume tri-dimensionnel : $\int_{\Omega} = \int \int \int r \, dr d\theta dz$. À nouveau, on peut omettre l'intégrale sur θ car aucune intégrande ne dépend de cette coordonnée.

2.3 Matrices locales

Les intégrales définies dans (26) sont nulles presque partout, sauf lorsque $\boldsymbol{\tau}_i$ et $\boldsymbol{\tau}_j$ sont non nulles sur le même élément Ω_e . Cela se produit lorsque i et j sont des nœuds de l'élément e . On assemble donc l'équation (27) en ajoutant les contributions de chaque élément e du maillage $\mathcal{T}_h$.

Sur un élément Ω_e avec $m = 3$ ou $m = 4$ nœuds, on définit donc les matrices locales $M^e, K^e \in \mathbb{R}^{2m \times 2m}$ qui lient les degrés de liberté q_i et q_j de l'élément e , $1 \leq i, j \leq m$. À

chaque paire de nœud (i, j) correspond donc un bloc 2×2 M_{ij}^e et K_{ij}^e liant les deux degrés de liberté (u, v) associés à ces nœuds.

$$M_{ij}^e = \begin{bmatrix} \langle \rho \boldsymbol{\varphi}_i \cdot \boldsymbol{\varphi}_j \rangle & \langle \rho \boldsymbol{\varphi}_i \cdot \boldsymbol{\psi}_j \rangle \\ \langle \rho \boldsymbol{\psi}_i \cdot \boldsymbol{\varphi}_j \rangle & \langle \rho \boldsymbol{\psi}_i \cdot \boldsymbol{\psi}_j \rangle \end{bmatrix} = \begin{bmatrix} \langle \rho \phi_i \phi_j \rangle & 0 \\ 0 & \langle \rho \phi_i \phi_j \rangle \end{bmatrix} \quad (28)$$

$$K_{ij}^e = \begin{bmatrix} \langle \boldsymbol{\epsilon}(\boldsymbol{\varphi}_i) \cdot \mathbf{C} \cdot \boldsymbol{\epsilon}(\boldsymbol{\varphi}_j) \rangle & \langle \boldsymbol{\epsilon}(\boldsymbol{\varphi}_i) \cdot \mathbf{C} \cdot \boldsymbol{\epsilon}(\boldsymbol{\psi}_j) \rangle \\ \langle \boldsymbol{\epsilon}(\boldsymbol{\psi}_i) \cdot \mathbf{C} \cdot \boldsymbol{\epsilon}(\boldsymbol{\varphi}_j) \rangle & \langle \boldsymbol{\epsilon}(\boldsymbol{\psi}_i) \cdot \mathbf{C} \cdot \boldsymbol{\epsilon}(\boldsymbol{\psi}_j) \rangle \end{bmatrix}. \quad (29)$$

Dans le code, on utilise la forme générale (29) pour plus de clarté et de modularité. Cela nous permet de changer le modèle constitutif sans avoir à modifier le code source. En contrepartie, on est forcé de calculer les deux produits matrices-vecteurs malgré les quelques entrées nulles.

Pour ce faire, il faut tout de même calculer les expressions de $\boldsymbol{\epsilon}(\boldsymbol{\tau}_i)$.

2.3.1 Cas plane

Dans le cas des tensions planes et des déformations planes, le terme $\epsilon_{zz}\sigma_{zz}$ de la double contraction disparaît, et on trouve grâce à (7) que

$$\boldsymbol{\epsilon}(\boldsymbol{\varphi}_i)_{\text{vec}} = \begin{bmatrix} \epsilon_{xx} \\ \epsilon_{yy} \\ 2\epsilon_{xy} \end{bmatrix} = \begin{bmatrix} \phi_{i,x} \\ 0 \\ \phi_{i,y} \end{bmatrix} \quad \boldsymbol{\epsilon}(\boldsymbol{\psi}_i)_{\text{vec}} = \begin{bmatrix} \epsilon_{xx} \\ \epsilon_{yy} \\ 2\epsilon_{xy} \end{bmatrix} = \begin{bmatrix} 0 \\ \phi_{i,y} \\ \phi_{i,x} \end{bmatrix}. \quad (30)$$

Ce qui nous permet d'écrire la matrice de raideur locale pour les déformations planes :

$$\begin{aligned} K_{ij}^e &= \int_{\Omega_e} \begin{bmatrix} \phi_{i,x} & 0 & \phi_{i,y} \\ 0 & \phi_{i,y} & \phi_{i,x} \end{bmatrix} \begin{bmatrix} \gamma & \lambda & 0 \\ \lambda & \gamma & 0 \\ 0 & 0 & \mu \end{bmatrix} \begin{bmatrix} \phi_{j,x} & 0 \\ 0 & \phi_{j,y} \\ \phi_{j,y} & \phi_{j,x} \end{bmatrix} d\Omega_e \\ &= \left[\begin{array}{c|c} \langle \phi_{i,x} \gamma \phi_{j,x} \rangle + \langle \phi_{i,y} \mu \phi_{j,y} \rangle & \langle \phi_{i,x} \lambda \phi_{j,y} \rangle + \langle \phi_{i,y} \mu \phi_{j,x} \rangle \\ \hline \langle \phi_{i,y} \lambda \phi_{j,x} \rangle + \langle \phi_{i,x} \mu \phi_{j,y} \rangle & \langle \phi_{i,y} \gamma \phi_{j,y} \rangle + \langle \phi_{i,x} \mu \phi_{j,x} \rangle \end{array} \right]. \end{aligned} \quad (31)$$

Dans le cas des tensions planes, il suffit de remplacer γ par α et λ par β dans (31).

2.3.2 Cas axisymétrique

Dans le cas axisymétrique, on doit également prendre en compte la composante $\epsilon_{\theta\theta}$ du tenseur de déformation comme mentionné à l'équation (15) :

$$\boldsymbol{\epsilon}(\boldsymbol{\varphi}_i)_{\text{vec}} = \begin{bmatrix} \epsilon_{rr} \\ \epsilon_{\theta\theta} \\ \epsilon_{zz} \\ 2\epsilon_{rz} \end{bmatrix} = \begin{bmatrix} \phi_{i,r} \\ \phi_{i,r}/r \\ 0 \\ \phi_{i,z} \end{bmatrix} \quad \boldsymbol{\epsilon}(\boldsymbol{\psi}_i)_{\text{vec}} = \begin{bmatrix} \epsilon_{rr} \\ \epsilon_{\theta\theta} \\ \epsilon_{zz} \\ 2\epsilon_{rz} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \phi_{i,z} \\ \phi_{i,r} \end{bmatrix}. \quad (32)$$

Il ne reste plus qu'à calculer la double contraction avec la matrice C et à intégrer en coordonnées cylindriques :

$$\begin{aligned}
K_{ij}^e &= \int_{\Omega_e} \begin{bmatrix} \phi_{i,r} & \phi_i/r & 0 & \phi_{i,z} \\ 0 & 0 & \phi_{i,z} & \phi_{i,r} \end{bmatrix} \begin{bmatrix} \gamma & \lambda & \lambda & 0 \\ \lambda & \gamma & \lambda & 0 \\ \lambda & \lambda & \gamma & 0 \\ 0 & 0 & 0 & \mu \end{bmatrix} \begin{bmatrix} \phi_{j,r} & 0 \\ \phi_j/r & 0 \\ 0 & \phi_{j,z} \\ \phi_{j,z} & \phi_{j,r} \end{bmatrix} d\Omega_e \\
&= \left[\begin{array}{c|c} \begin{aligned} &\langle \phi_{i,r} r \gamma \phi_{j,r} \rangle + \langle \phi_{i,z} r \mu \phi_{j,z} \rangle \\ &+ \langle \phi_{i,r} \lambda \phi_j \rangle + \langle \phi_i \lambda \phi_{j,r} \rangle \\ &\quad \langle \phi_i \gamma/r \phi_j \rangle \end{aligned} & \begin{aligned} &\langle \phi_{i,r} r \lambda \phi_{j,z} \rangle + \langle \phi_{i,z} r \mu \phi_{j,r} \rangle \\ &+ \langle \phi_i \lambda \phi_{j,z} \rangle \end{aligned} \\ \hline \begin{aligned} &\langle \phi_{i,z} r \lambda \phi_{j,r} \rangle + \langle \phi_{i,r} r \mu \phi_{j,z} \rangle \\ &+ \langle \phi_{i,z} \lambda \phi_j \rangle \end{aligned} & \begin{aligned} &\langle \phi_{i,z} r \gamma \phi_{j,z} \rangle + \langle \phi_{i,r} r \mu \phi_{j,r} \rangle \end{aligned} \end{array} \right]. \quad (33)
\end{aligned}$$

On remarque que les termes en noir sont similaires au cas plane alors que des contributions supplémentaires apparaissent en rouge. Ces contributions sont dues à la présence d'une déformation tangentielle $\epsilon_{\theta\theta}$ malgré l'absence de déplacement dans cette direction.

2.3.3 Terme source local

Le terme source est lui bien plus simple à assembler :

$$b_i^e = \begin{bmatrix} \langle \boldsymbol{\varphi}_i \cdot \mathbf{f} \rangle \\ \langle \boldsymbol{\psi}_i \cdot \mathbf{f} \rangle \end{bmatrix} = \begin{cases} \begin{bmatrix} \langle \phi_i f_1 \rangle \\ \langle \phi_i f_2 \rangle \end{bmatrix} & \text{cas plane} \\ \begin{bmatrix} \langle \phi_i r f_r \rangle \\ \langle \phi_i r f_z \rangle \end{bmatrix} & \text{cas axisymétrique} \end{cases} \quad (34)$$

2.4 Conditions naturelles de Neumann et Robin

Ces conditions sont appelées *naturelles* car elles apparaissent naturellement dans la formulation faible. Elles découlent de la contrainte imposée sur la frontière du solide :

$$\mathbf{n} \cdot \boldsymbol{\sigma}|_{\Gamma} = \mathbf{t} \quad \forall \mathbf{x} \in \Gamma_N \quad (35)$$

$$\mathbf{n} \cdot \boldsymbol{\sigma}|_{\Gamma} = \mathbf{t} - k(\mathbf{u} - \mathbf{u}_0) \quad \forall \mathbf{x} \in \Gamma_R. \quad (36)$$

- Dans le cas de la condition de Neumann, on connaît la contrainte au bord.
- Dans le cas de la condition de Robin, on fait l'hypothèse que le bord est attaché à un ressort de raideur k autour d'un point de référence $\mathbf{x} + \mathbf{u}_0$.

On voit donc clairement que la condition de Neumann est un cas particulier de la condition de Robin, lorsque $k = 0$. Dans la suite, nous allons donc exclusivement nous intéresser à la condition de Robin sous la forme

$$\mathbf{n} \cdot \boldsymbol{\sigma}|_{\Gamma} = \mathbf{p} - k\mathbf{u}. \quad (37)$$

Le terme de surface de l'équation (22) s'écrit alors

$$\int_{\Gamma} (\mathbf{n} \cdot \boldsymbol{\sigma}) \cdot \mathbf{v} \, d\Gamma. \quad (38)$$

Pour appliquer cette condition dans la direction normale ou tangentielle uniquement, il faut décomposer chaque vecteur dans ces deux directions. N'importe quelle autre paire de direction orthogonales est également possible.

$$\begin{aligned} & \int_{\Gamma} [(\mathbf{n} \cdot \boldsymbol{\sigma} \cdot \mathbf{n})\mathbf{n} + (\mathbf{n} \cdot \boldsymbol{\sigma} \cdot \mathbf{t})\mathbf{t}] \cdot [(\mathbf{v} \cdot \mathbf{n})\mathbf{n} + (\mathbf{v} \cdot \mathbf{t})\mathbf{t}] \, d\Gamma \\ &= \int_{\Gamma} (p_n - k_n \mathbf{u} \cdot \mathbf{n})(\boldsymbol{\tau} \cdot \mathbf{n}) \, d\Gamma + \int_{\Gamma} (p_t - k_t \mathbf{u} \cdot \mathbf{t})(\boldsymbol{\tau} \cdot \mathbf{t}) \, d\Gamma \end{aligned} \quad (39)$$

La modification à apporter au système $Kq = b$ pour une condition de Robin dans la direction $\mathbf{n}$ s'obtient en utilisant successivement $\boldsymbol{\tau} = \boldsymbol{\varphi}_i = \phi_i \mathbf{e}_1$ et $\boldsymbol{\tau} = \boldsymbol{\psi}_i = \phi_i \mathbf{e}_2$:

$$K_{ij} \leftarrow K_{ij} + \begin{bmatrix} \langle\langle k_n n_1 \phi_j \phi_i n_1 \rangle\rangle & \langle\langle k_n n_2 \phi_j \phi_i n_1 \rangle\rangle \\ \langle\langle k_n n_1 \phi_j \phi_i n_2 \rangle\rangle & \langle\langle k_n n_2 \phi_j \phi_i n_2 \rangle\rangle \end{bmatrix}, \quad (40)$$

$$b_i \leftarrow b_i + \begin{bmatrix} \langle\langle p_n \phi_i n_1 \rangle\rangle \\ \langle\langle p_n \phi_i n_2 \rangle\rangle \end{bmatrix} \quad (41)$$

La modification pour la direction $\mathbf{t}$ s'obtient en remplaçant $\mathbf{n}$, k_n et p_n par $\mathbf{t}$, k_t et p_t . L'extension au cas axisymétrique est immédiate : il suffit d'ajouter un facteur r dans les intégrales de surface.

2.5 Conditions essentielles de Dirichlet

Ces conditions sont appelées *essentiellles* car elles doivent être imposées à la formulation discrète en modifiant la matrice de raideur.

On souhaite pouvoir imposer à notre solide un déplacement dans une seule direction en lui laissant la liberté de se déplacer dans les autres directions (l'autre direction en 2D). Dans le cas d'une pièce qui repose sur une surface plane, on veut par exemple interdire tout déplacement normal à cette surface mais lui laisser la liberté de glisser dessus.

Dans le cas où l'on souhaite imposer le déplacement complet, il suffit de le faire successivement dans chaque direction.

2.5.1 Dans une direction alignée avec un axe

Comme précisé précédemment, sur la frontière de Dirichlet Γ_D , les fonctions tests $\boldsymbol{\tau}$ dans la direction du déplacement imposé sont nulles. Prenons par exemple un déplacement $\bar{u}$ imposé selon la direction $\mathbf{e}_x$ au nœud i . Il faut alors effectuer les opérations suivantes sur le degré

de liberté p correspondant

$$\begin{aligned}
K_{pq} &\leftarrow 0 & \forall q \neq p & \iff \varphi_i = \mathbf{0} \\
K_{pp} &\leftarrow 1 \\
b_p &\leftarrow \bar{u} \\
K_{qp} &\leftarrow 0 & \forall q \neq p \\
b_q &\leftarrow b_q - K_{qp}\bar{u} & \forall q \neq p
\end{aligned} \tag{42}$$

Les deux dernières lignes ne sont pas obligatoires, mais elles permettent de maintenir la symétrie de la matrice de raideur.

2.5.2 Dans une direction arbitraire

Disons que l'on souhaite imposer au nœud i un déplacement $\bar{u}$ dans une direction $\mathbf{n}$ qui forme avec $\mathbf{t}$ une base orthonormée. Il faut alors modifier à la fois les inconnues du nœud i et les fonctions tests.

Soit $Q_{\text{loc}} \in \mathbb{R}^{2 \times 2}$, la matrice qui transforme un déplacement normal / tangent vers un déplacement aligné avec $\mathbf{e}_1$ / $\mathbf{e}_2$:

$$\begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \underbrace{\begin{bmatrix} n_1 & t_1 \\ n_2 & t_2 \end{bmatrix}}_{Q_{\text{loc}}} \begin{bmatrix} u_n \\ u_t \end{bmatrix} \iff \begin{bmatrix} u_n \\ u_t \end{bmatrix} = \underbrace{\begin{bmatrix} n_1 & n_2 \\ t_1 & t_2 \end{bmatrix}}_{Q_{\text{loc}}^*} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \tag{43}$$

Étendons Q_{loc} à une matrice $Q \in \mathbb{R}^{2n \times 2n}$ dont seul le bloc 2×2 associé au nœud i diffère de l'identité :

$$q = \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_i \\ v_i \\ \vdots \end{bmatrix} = Qz \iff z = \begin{bmatrix} u_1 \\ v_1 \\ \vdots \\ u_n \\ v_t \\ \vdots \end{bmatrix} = Q^*q \iff Q = \begin{bmatrix} 1 & 0 & & & \\ 0 & 1 & & & \\ & & \ddots & & \\ & & & n_1 & t_1 \\ & & & n_2 & t_2 \\ & & & & \ddots \end{bmatrix} \tag{44}$$

Il faut ensuite "tourner" la matrice de raideur K et le vecteur de forçage b pour les ramener dans le bon repère :

$$Kq = b \longrightarrow Q^*KQz = Q^*b \tag{45}$$

On peut ensuite appliquer sur ce nouveau système la procédure présentée à la section 2.5.1 pour imposer le déplacement $\bar{u}$ au nœud i dans la direction $\mathbf{n}$. Enfin, pour passer de la solution z à la solution q , il faut appliquer la transformation inverse, de part et d'autre pour maintenir la symétrie. C'est en pratique ce qui est implémenté dans le code source.

Essayons néanmoins de décrire la procédure de manière plus formelle. Définissons la matrice $C = I - \mathbf{e}_p \mathbf{e}_p^*$, une matrice de projection dont seule la première entrée du bloc 2×2 associé au nœud i diffère de l'identité.

$$Kx = b \quad (46)$$

$$\longrightarrow Q^* K Q z = Q^* b \quad \text{Rotation} \quad (47)$$

$$\longrightarrow C Q^* K Q z = C Q^* b \quad \text{Annulation} \quad (48)$$

$$\longrightarrow (C Q^* K Q C + \mathbf{e}_p \mathbf{e}_p^*) z = C Q^* b - C Q^* K Q \mathbf{e}_p \bar{u} + \mathbf{e}_p \bar{u} \quad \text{Symétrisation} \quad (49)$$

$$\longrightarrow Q(C Q^* K Q C + \mathbf{e}_p \mathbf{e}_p^*) Q^* x = Q C Q^* b - Q C Q^* K Q \mathbf{e}_p \bar{u} + Q \mathbf{e}_p \bar{u} \quad \text{Rotation}' \quad (50)$$

Le système ci-dessus peut être habilement réécrit comme

$$(N + T K T) x = T b - T K \tilde{n} \bar{u} + \tilde{n} \bar{u}, \quad \text{avec} \quad (51)$$

$$\begin{aligned} T \triangleq Q C Q^* = & \left[\begin{array}{cccc} 1 & 0 & & \\ 0 & 1 & & \\ & & \ddots & \\ & & & t_1^2 & t_1 t_2 \\ & & & t_1 t_2 & t_2^2 \\ & & & & & \ddots \end{array} \right] & \left| \right. & N \triangleq Q \mathbf{e}_p \mathbf{e}_p^* Q^* = & \left[\begin{array}{cccc} 0 & 0 & & \\ 0 & 0 & & \\ & & \ddots & \\ & & & n_1^2 & n_1 n_2 \\ & & & n_1 n_2 & n_2^2 \\ & & & & & \ddots \end{array} \right] & \left| \right. & \tilde{n} \triangleq Q \mathbf{e}_p \\ & & & & & & & & = & \begin{bmatrix} 0 \\ 0 \\ \vdots \\ n_1 \\ n_2 \\ \vdots \end{bmatrix} \end{aligned} \quad (52)$$

On constate fort heureusement qu'avec $n_1 = 1 = -t_2$ et $n_2 = 0 = t_1$, on retombe sur le cas particulier de la section 2.5.1.

2.6 Calcul des contraintes

Lorsque le système est résolu, on obtient le déplacement $\mathbf{u}_j$ pour chaque nœud j . On peut ensuite calculer aux points d'intégration les déformations grâce à la relation (2) et enfin les contraintes grâce au modèle constitutif (3).

Étant donné qu'on utilise des éléments linéaires triangulaires ($\mathcal{P}_1$) ou quadrangulaires ($\mathcal{Q}_1$), les déformations $\boldsymbol{\epsilon}^h$ sont discontinues par éléments. Par conséquent les contraintes $\boldsymbol{\sigma}^h$ le sont également. Il est toutefois possible de reconstruire une approximation $\boldsymbol{\sigma}^c$ continue aux nœuds du maillage :

$$\boldsymbol{\sigma}^c(\mathbf{x}) = \sum_{j=1}^n \phi_j(\mathbf{x}) \boldsymbol{\sigma}_j^h. \quad (53)$$

Au moins deux méthodes sont possibles :

- Moyenner autour de chaque nœud les contraintes des éléments adjacents, ou bien
- Minimiser l'intégrale du carré de la différence entre les contraintes discontinues $\boldsymbol{\sigma}^h$ et les contraintes continues $\boldsymbol{\sigma}^c$.

2.6.1 Moyenne pondérée

Cette première méthode a l'avantage d'être rapide, mais fournit de moins bons résultats. En notant σ_j^c la valeur nodale du champ de contrainte et D_j l'ensemble des éléments adjacents au nœud j , on a alors

$$\sigma_j^c = \frac{\int_{D_j} \phi_j \sigma^h d\mathbf{x}}{\int_{D_j} \phi_j d\mathbf{x}}. \quad (54)$$

2.6.2 Moindre carrés

Cette seconde méthode fournit de meilleurs résultats mais se révèle plus coûteuse étant donné qu'il faut résoudre un système linéaire. On cherche ici à minimiser l'intégrale

$$J(\sigma_1, \dots, \sigma_n) = \int_{\Omega} \left(\sigma^c(\sigma_1, \dots, \sigma_n) - \sigma^h \right)^2 d\Omega \quad (55)$$

pour chaque composante σ^c et σ^h du tenseur de contrainte.

On trouve les paramètres σ_i du minimiseur σ^c grâce à la condition de stationnarité

$$\begin{aligned} \partial_{\sigma_i} J &= 2 \int_{\Omega} \left(\sigma^c(\sigma_1, \dots, \sigma_n) - \sigma^h \right) \partial_{\sigma_i} \sigma^c d\Omega = 0 \\ \Rightarrow \int_{\Omega} \left(\sum_j \phi_j \sigma_j - \sigma^h \right) \phi_i d\Omega &= 0 \\ \Rightarrow \sum_j \left[\int_{\Omega} (\phi_j \cdot \phi_i) d\Omega \right] \sigma_j &= \int_{\Omega} (\sigma^h \cdot \phi_i) d\Omega \quad \forall i = 1 \dots n. \end{aligned} \quad (56)$$

On retrouve une matrice de masse similaire à celle définie dans (28), hormis que celle-ci ne porte que sur un champ scalaire. L'opération $M\sigma = b$ présentée ci-dessus n'est en fait qu'une projection de la contrainte σ^h sur l'espace $\mathcal{U}^h$.

2.7 Analyse modale

En faisant l'hypothèse qu'on cherche une solution harmonique $\mathbf{u} = \Phi e^{i\omega t}$, on peut réécrire l'équation matricielle (27) sans forçage b sous la forme d'un problème aux valeurs propres généralisé :

$$K\Phi = \omega^2 M\Phi, \quad (57)$$

dont les solutions non-triviales (ω_i^2, Φ_i) satisfont

$$\Phi_i K \Phi_j = \omega_i^2 \delta_{ij} \quad \text{et} \quad (58)$$

$$\Phi_i M \Phi_j = \delta_{ij}. \quad (59)$$

C'est à dire que les vecteurs propres Φ_i ne sont pas orthonormés au sens classique mais à travers la matrice de masse. Il est possible de le démontrer grâce à la symétrie des matrices K et M :

$$\left\{ \begin{array}{l} \Phi_i^* K \Phi_j = \omega_i^2 \Phi_i^* M \Phi_j \\ \Phi_i^* K \Phi_j = \Phi_j^* K \Phi_i = \omega_j^2 \Phi_j^* M \Phi_i = \omega_j^2 \Phi_i^* M \Phi_j \end{array} \right\} \implies (\omega_i^2 - \omega_j^2) \Phi_i^* M \Phi_j = 0. \quad (60)$$

- Si $\omega_i \neq \omega_j$, alors $\Phi_i^* M \Phi_j = 0$.
- Si $\omega_i = \omega_j$, alors on peut M -orthonormaliser les vecteurs propres dans l'espace propre associé.

On trouve enfin $\Phi_i^* K \Phi_j = \omega_i^2 \Phi_i^* M \Phi_j = \omega_i^2 \delta_{ij}$.

Les fréquences naturelles du solide sont simplement obtenues par $f_i = \omega_i/2\pi$ [Hz].

3 Résolution numérique

3.1 Système linéaire

Comme dans n'importe quel problème d'élément finis, la matrice de raideur du système $Kx = b$ (27) est particulièrement creuse. En effet, chaque degré de liberté q_i communique avec q_j uniquement si un élément les contient tous les deux. Le nombre d'entrées non-nulles (**nnz**) par ligne est donc en moyenne de

- $2 \cdot (6 + 1) = 14$ pour une triangulation
- $2 \cdot (8 + 1) = 18$ pour une quadrangulation

Deux types de solveurs sont donc proposés pour tirer profit de cet aspect creux :

1. solveur direct où K est stocké sous format bande,
2. solveur itératif où K est stocké sous format CSR.

3.1.1 Solveur direct – LDL'

Il s'agit ici simplement d'une factorisation de Cholesky de la matrice de raideur K :

$$K = LDL^T \tag{61}$$

où L est une matrice triangulaire inférieure de diagonale unité et D est une matrice diagonale.

Cette factorisation, réalisée *in-place*, génère de nombreuses entrées non-nulles par rapport à la matrice de raideur d'origine. Il est possible de réduire ce *fill-in* en permutant les lignes et colonnes de K . On distingue généralement deux grandes familles de stratégies, selon qu'elles visent à réduire la bande (et indirectement le *fill-in*), ou qu'elles ciblent directement la réduction du *fill-in* lors de l'élimination.

- **Réduction de bande.** Ces méthodes, comme le tri des nœuds par coordonnées ou l'algorithme de *Reverse Cuthill-McKee* (RCMK) basé sur un parcours en largeur du maillage (*breadth-first search*), visent à concentrer les entrées non-nulles autour de la diagonale. RCMK, bien qu'un peu plus complexe à implémenter, offre de bien meilleurs résultats dès que la géométrie n'est plus triviale.
- **Réduction directe du *fill-in*.** Les méthodes comme *Minimum Degree* ou *Nested Dissection* minimisent plus efficacement les nouvelles entrées créées, mais requièrent des structures de données plus sophistiquées. Le *fill-in* se répartit alors sur l'ensemble de la matrice.

Dans le code, l'algorithme de RCMK a été choisi puisqu'il offre un bon compromis entre simplicité d'implémentation et efficacité.

Lorsque la matrice K a une bande de largeur b , cette factorisation s'effectue en $\mathcal{O}(nb^2)$. Le système $Kx = LDL^T x = b$ est alors résolu en trois étapes :

$$Lz = b \quad \longrightarrow \quad Dy = z \quad \longrightarrow \quad L^T x = y \quad (62)$$

3.1.2 Solveur itératif – Gradient conjugué

Le solveur itératif implémenté est l'algorithme du gradient conjugué (CG), conçu pour résoudre des systèmes symétriques définis positifs. À la différence des méthodes directes, cet algorithme démarre d'une solution initiale x_0 et converge progressivement vers la solution exacte x^* en plusieurs itérations. Il est donc particulièrement adapté aux systèmes de grande taille, où les méthodes directes deviennent trop coûteuses en temps et en mémoire.

Le coût d'une itération de l'algorithme CG est globalement dominé par le produit matrice-vecteur Kx_i . Il est donc essentiel de choisir une représentation de K qui permette de calculer efficacement ce produit : le format **CSR** (*Compressed Sparse Row*). Le coût de ce produit est alors seulement de $\mathcal{O}(\text{nnz})$.

Il est possible de montrer que l'erreur $e_i = x_i - x^*$ à l'itération i est bornée par l'erreur initiale e_0 :

$$\|e_i\|_K \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^i \|e_0\|_K, \quad (63)$$

où κ est le nombre de condition de la matrice K , défini comme

$$\kappa(K) = \frac{\lambda_{\max}(K)}{\lambda_{\min}(K)}. \quad (64)$$

On peut donc obtenir le nombre d'itérations N nécessaires pour borner l'erreur par ϵ :

$$\begin{aligned} \|e_N\|_K &\leq \epsilon \\ 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^N \|e_0\|_K &\leq \epsilon \\ \left(1 - \frac{2}{\sqrt{\kappa}} \right)^N &\leq \frac{\epsilon}{2\|e_0\|_K} \\ \left(1 - \frac{2N/\sqrt{\kappa}}{N} \right)^N &\leq \frac{\epsilon}{2\|e_0\|_K} \\ \exp \left(-\frac{2N}{\sqrt{\kappa}} \right) &\leq \frac{\epsilon}{2\|e_0\|_K} \\ \frac{1}{2} \sqrt{\kappa(K)} \ln (2\|e_0\|_K / \epsilon) &\leq N. \end{aligned} \quad (65)$$

Quelle que soit la dimension, le nombre de condition κ augmente avec $\sim \mathcal{O}(h^{-2})$, où h est la taille de maille. À partir d'un certain niveau de raffinement, le nombre d'itérations N devient prohibitif. Il devient donc nécessaire d'utiliser un préconditionneur pour réduire le κ .

L'idée est de transformer le système $Kx = b$ en un système équivalent

$$M^{-1}Kx = M^{-1}b, \quad (66)$$

qui doit présenter deux propriétés :

- $\kappa(M^{-1}K)$ doit être plus faible que celui de K ,
- M doit être facilement inversible.

Pour utiliser CG, il faut que la matrice soit SDP, or $M^{-1}K$ ne l'est pas en général. Il faut donc supposer que $M^{-1} = EE^T$ avec E non singulière. On peut alors écrire

$$\begin{aligned} M^{-1}Kx &= M^{-1}b \\ E^T Kx &= E^T b \\ (E^T K E)(E^{-1}x) &= E^T b \\ \hat{K}\hat{x} &= \hat{b}. \end{aligned} \quad (67)$$

Avec un peu d'algèbre, l'algorithme du gradient conjugué préconditionné (PCG) se ramène à un CG augmenté d'une résolution de système linéaire $Mz = r$ à chaque itération.

Dans le code, plusieurs préconditionneurs sont implémentés :

- **Jacobi** : $M = D$ avec D la diagonale de K ,
- **SSOR** : $M = (D + L)D^{-1}(D + L)^T$ avec L la matrice triangulaire inférieure de K ,
- **ILU(0)** : $M = LDL^T$ avec L la factorisation incomplète de Cholesky de K où l'on n'autorise aucun *fill-in*,
- **ILU(1)** : $M = LDL^T$ avec L la factorisation incomplète de Cholesky de K où l'on n'autorise qu'un degré de *fill-in*.

3.2 Mode propre

Pour calculer les modes propres du système, il faut résoudre le problème aux valeurs propres généralisé (57) $K\Phi = \omega^2 M\Phi$. Il existe plusieurs méthodes pour résoudre ce type de problème. Vu qu'on ne cherche que les quelques modes de faible fréquence, la méthode la plus adaptée est l'itération inverse combinée à une déflation.

On pourrait être tentés de d'inverser le K ou M pour retomber sur un problème aux valeurs propres standard. Cependant, ces matrices sont généralement très grandes et creuses, et leur inversion est donc prohibitive. Pour obtenir ces fameux modes, il est en fait possible de réaliser une itération de la puissance sur $K^{-1}M$ sans avoir à l'inverser :

$$x_k \leftarrow K^{-1}Mx_{k-1} \iff \left\{ \begin{array}{l} z_k \leftarrow Mx_{k-1} \\ x_k \leftarrow \text{solve}(K, z_k) \\ \lambda_k \leftarrow \frac{x_k^* K x_k}{x_k^* M x_k} \end{array} \right\}. \quad (68)$$

Il faut donc factoriser K **une seule fois** au début de l'algorithme, en $\mathcal{O}(nb^2)$. À chaque itération, il faut ensuite seulement calculer un produit matrice-vecteur et résoudre un système linéaire, chacun en $\mathcal{O}(nb)$.

3.2.1 Déflation

Nous souhaitons trouver les N premiers modes propres du système. Or l'itération inverse converge uniquement vers le mode propre Φ_i associé à

$$\lambda_{\max}(K^{-1}M) = \lambda_{\min}(KM^{-1}) \quad (69)$$

On peut le montrer en décomposant l'itéré initial x_0 dans la base propre de KM^{-1} . On observe alors que l'itération inverse converge vers le mode propre associé à la valeur propre minimale $\lambda_1 = \omega_1^2$:

$$x_0 = \sum_{j=1}^n \alpha_j \Phi_j \quad \text{avec } \lambda_1 < \lambda_2 < \dots < \lambda_{n-1} < \lambda_n \quad (70)$$

$$\implies x_k = (K^{-1}M)^k x_0 = \sum_{j=1}^n \alpha_j (K^{-1}M)^k \Phi_j = \sum_{j=1}^n \alpha_j \frac{1}{\lambda_j^k} \Phi_j \quad (71)$$

$$\implies x_k \rightarrow \alpha_1 \lambda_1^{-k} \Phi_1 \quad \text{si } \alpha_1 \neq 0 \quad (72)$$

$$(73)$$

On constate donc qu'il est possible de trouver le second mode propre si l'itéré initial x_0 ne contient pas de composante $\alpha_1 \Phi_1$. Lorsque ce premier mode propre est trouvé, il faut donc le "supprimer" d'un itéré initial w pour converger vers le second mode propre. Le coefficient α_1 , nécessaire pour y parvenir est obtenu grâce à une projection M -orthogonale :

$$w = \sum_{j=1}^n \alpha_j \Phi_j \implies \Phi_i^* M w = \sum_{j=1}^n \alpha_j \Phi_i^* M \Phi_j = \sum_{j=1}^n \alpha_j \delta_{ij} = \alpha_i \quad (74)$$

où l'on suppose que Φ_i est normalisé au sens de M , i.e. $\Phi_i^* M \Phi_i = 1$. Lorsque l'on connaît les m premiers modes propres, l'itéré initial pour trouver le mode propre $(m+1)$ est donc

$$x_0^{(m+1)} = w - \sum_{i=1}^m \alpha_i \Phi_i = \left(I - \sum_{i=1}^m \Phi_i \Phi_i^* M \right) w. \quad (75)$$

En pratique, à cause des erreurs d'arrondis, les modes déflatés Φ_i avec $i \leq m$ ont tendance à réapparaître dans les itérés suivants x_k . Il faut donc répéter la projection M -orthogonale à chaque itération.

La procédure est présentée en détails dans l'algorithme 1 ci-dessous.

Algorithm 1 Itération inverse avec déflation pour un problème généralisé $K\Phi = \omega^2 M\Phi$

```
1: Input: Matrices  $K, M$ , number of modes  $N$ 
2: Factorize  $K$  into  $LDL^T$ 
3: for  $i = 1$  to  $N$  do ▷ Modes
4:    $x_0 \leftarrow$  random vector
5:   for  $k = 1, 2, \dots$  do ▷ Itérations
6:      $z_k \leftarrow Mx_{k-1}$ 
7:      $x_k \leftarrow \text{solve}(K, z_k)$ 
8:     for  $j = 1$  to  $i - 1$  do ▷ Orthogonalisation
9:        $\alpha \leftarrow \Phi_j^* Mx_k$ 
10:       $x_k \leftarrow x_k - \alpha \Phi_j$  ▷ Gram-Schmidt modifié
11:    end for
12:     $x_k \leftarrow x / \sqrt{x_k^* Mx_k}$  ▷ Normalisation
13:     $\nu_k \leftarrow x_k^* Kx_k$ 
14:  end for
15:   $\Phi_i \leftarrow x_k$  ▷ Mode propre
16:   $\lambda_i \leftarrow \nu_k$  ▷ Valeur propre
17:  Return:  $\Phi_i, \lambda_i$ 
18: end for
```

3.2.2 Shift

À l'instar de l'itération inverse classique, il est possible de *shifter* le spectre du système pour trouver le mode propre autour d'une fréquence cible. Soustraire μM à la matrice de raideur K revient à décaler le spectre des valeurs propres d'une quantité μ :

$$(K - \mu M)\Phi = K\Phi - \mu M\Phi = (\lambda - \mu)M\Phi \quad (76)$$

Si μ est proche d'une valeur propre λ_i , alors le système inverse $(K - \mu M)^{-1}M$ possède une valeur propre $(\lambda_i - \mu)^{-1} \gg (\lambda_j - \mu)^{-1}$ pour $j \neq i$. L'itération inverse

$$x_k \leftarrow (K - \mu M)^{-1}Mx_{k-1} \iff \left\{ \begin{array}{l} z_k \leftarrow Mx_{k-1} \\ x_{k+1} \leftarrow \text{solve}(K - \mu M, z_k) \end{array} \right\} \quad (77)$$

converge donc très vite vers le mode propre Φ_i associé à cette fréquence propre $\lambda_i = \omega_i^2$.

3.3 Adimensionalisation

Plusieurs grandeurs dimensionnelles sont présentes dans le système :

- la masse volumique ρ [kg/m³],
- le module d'élasticité E [Pa],
- la longueur caractéristique L [m]

Sur base de ces paramètres, on peut définir un temps caractéristique

$$T = L\sqrt{\frac{\rho}{E}} \quad [\text{s}]. \quad (78)$$

En notant d'un tilde les grandeurs dimensionnelles et sans tilde les grandeurs adimensionnelles, on peut réécrire la formulation faible (22)

$$\int_{\tilde{\Omega}} \rho \frac{\partial^2 \tilde{\mathbf{u}}}{\partial \tilde{t}^2} \cdot \tilde{\mathbf{v}} \, d\tilde{\mathbf{x}} + \int_{\tilde{\Omega}} \tilde{\boldsymbol{\sigma}}(\tilde{\mathbf{u}}) : \tilde{\boldsymbol{\epsilon}}(\tilde{\mathbf{v}}) \, d\tilde{\mathbf{x}} = \int_{\tilde{\Omega}} \tilde{\mathbf{f}} \cdot \tilde{\mathbf{v}} \, d\tilde{\mathbf{x}} + \int_{\tilde{\Gamma}} \tilde{\mathbf{g}} \cdot \tilde{\mathbf{v}} \, d\tilde{\mathbf{s}} \quad (79)$$

$$\int_{\Omega} \frac{\rho L^2 E}{\rho L^2} \frac{\partial^2 \mathbf{u}}{\partial t^2} \cdot \mathbf{v} \, d\mathbf{x} + \int_{\Omega} E \boldsymbol{\sigma}(\mathbf{u}) : \boldsymbol{\epsilon}(\mathbf{v}) \, d\mathbf{x} = \int_{\Omega} \frac{E}{L} \mathbf{f} \cdot \mathbf{v} L \, d\mathbf{x} + \int_{\Gamma} E \mathbf{g} \cdot \mathbf{v} L \, d\mathbf{s} / L \quad (80)$$

$$\int_{\Omega} \frac{\partial^2 \mathbf{u}}{\partial t^2} \cdot \mathbf{v} \, d\mathbf{x} + \int_{\Omega} \boldsymbol{\sigma}(\mathbf{u}) : \boldsymbol{\epsilon}(\mathbf{v}) \, d\mathbf{x} = \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\mathbf{x} + \int_{\Gamma} \mathbf{g} \cdot \mathbf{v} \, d\mathbf{s} \quad (81)$$

On observe donc que tout paramètre physique a disparu de l'équation lorsque les grandeurs sont adimensionnelles. Les liens entre grandeurs dimensionnelles et adimensionnelles sont résumés ici :

$$\begin{aligned} \tilde{\mathbf{x}} &= L\mathbf{x} & \tilde{\mathbf{u}} &= L\mathbf{u} & \tilde{\boldsymbol{\sigma}} &= E\boldsymbol{\sigma} & \tilde{\mathbf{f}} &= \frac{E}{L}\mathbf{f} \\ \tilde{t} &= L\sqrt{\frac{\rho}{E}} t & \tilde{\mathbf{v}} &= L\mathbf{v} & \tilde{\boldsymbol{\epsilon}}(\tilde{\mathbf{u}}) &= \boldsymbol{\epsilon}(\mathbf{u}) & \tilde{\mathbf{g}} &= E\mathbf{g}. \end{aligned} \quad (82)$$

En pratique, dans le code source, on scale les coordonnées $\mathbf{x}$ par une grandeur L choisie en fonction de la géométrie du solide. Le système est ensuite assemblé sous sa forme adimensionnelle grâce aux relations (82).

Lorsque le système adimensionnel $Kx = b$ est résolu, on retrouve les déplacements et contraintes dimensionnels en se référant à nouveau aux relations (82).

Concernant les fréquences naturelles ω_i , on les obtient sur base des valeurs propres adimensionnelles λ_i :

$$\omega_i = \frac{\sqrt{\lambda_i}}{T} = \sqrt{\frac{\lambda_i E}{\rho L^2}} \quad [\text{rad/s}]. \quad (83)$$

4 Validation

4.1 Poutre encastrée

Une poutre encastrée de longueur L , de section rectangulaire de hauteur h et largeur b , de densité ρ et de raideur E est soumise à son propre poids.

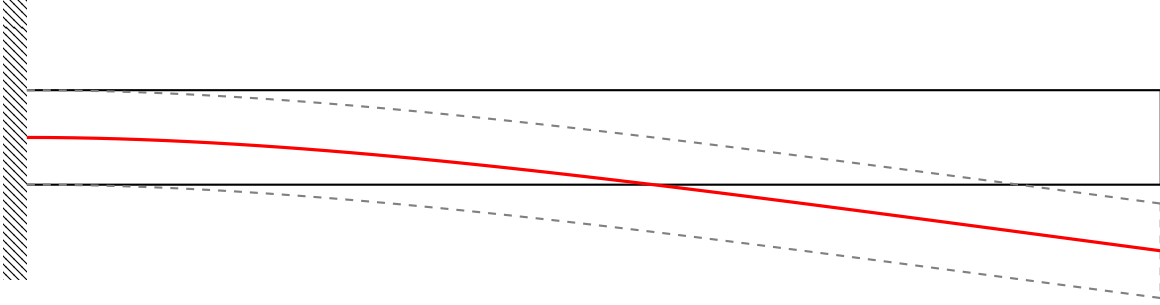


Figure 1. Poutre encastrée-libre, selon la théorie d'Euler-Bernoulli.

4.1.1 Flexion

La flèche de la poutre soumise à une distribution de force verticale $q(x)$ satisfait

$$\partial_x^4 v(x) = \frac{q(x)}{EI} \quad (84)$$

où le moment d'inertie de la section de la poutre $I = bh^3/12$ dans notre cas. On trouve donc pour notre poutre encastrée :

$$v(x) = \frac{qL^4}{24EI} \frac{x^2}{L^2} \left(6 - 4\frac{x}{L} + \frac{x^2}{L^2} \right) = \frac{-\rho g L^4}{2Eh^2} \frac{x^2}{L^2} \left(6 - 4\frac{x}{L} + \frac{x^2}{L^2} \right). \quad (85)$$

4.1.2 Vibration

La vibration verticale d'une poutre droite satisfait l'équation

$$\rho S \partial_t^2 v(x) = -EI \partial_x^4 v(x) \quad (86)$$

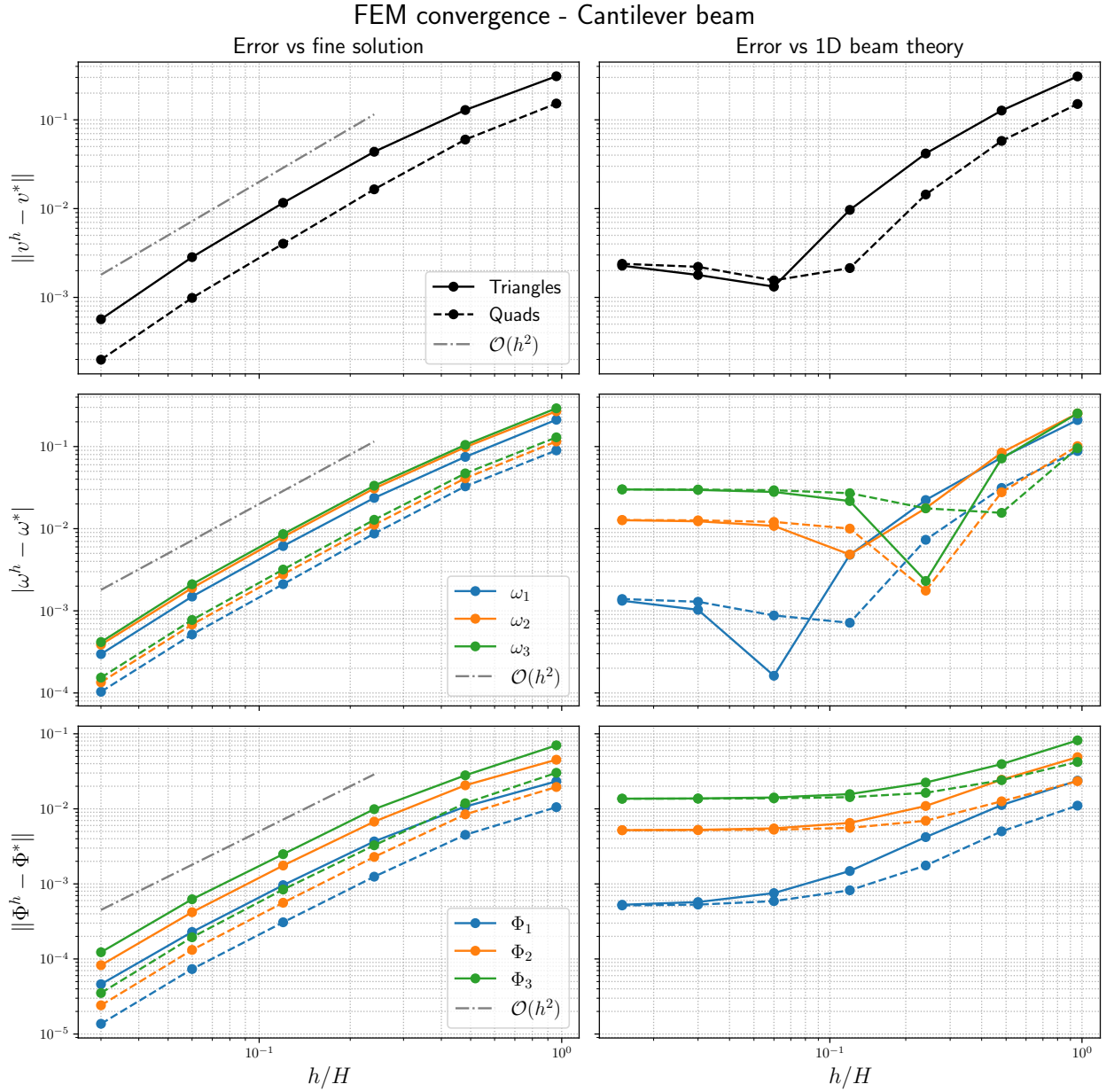
où S est l'aire de la section de la poutre. Avec nos conditions frontières, on trouve les modes propres $\Phi_i(x)$ et les fréquences naturelles ω_i associées :

$$\begin{aligned} v(x, t) &= \sum_{i=1}^{\infty} \alpha_i \Phi_i(x) \cos(\omega_i t + \phi_i) \\ \omega_i &= \frac{\gamma_i^2}{L^2} \sqrt{\frac{EI}{\rho S}} = \frac{\gamma_i^2}{L^2} \sqrt{\frac{Eh^2}{12\rho}} \\ -1 &= \cos(\gamma) \cosh(\gamma) \implies \gamma \approx [1.875, 4.694, 7.855, 10.996 \dots] \\ \Phi_i(x) &= (\sin \gamma_i - \sinh \gamma_i)(\sin(\gamma_i \xi) - \sinh(\gamma_i \xi)) \\ &\quad + (\cos \gamma_i + \cosh \gamma_i)(\cos(\gamma_i \xi) - \cosh(\gamma_i \xi)) \quad \text{où } \xi = x/L. \end{aligned} \quad (87)$$

4.1.3 Analyse de convergence

L'analyse de convergence du code a été réalisée sur une poutre encastree de longueur $L = 20$ m, de section rectangulaire de hauteur $H = 1$ m, avec $E = 211$ GPa, $\rho = 7850$ kg/m³, $\nu = 0.3$, $g = 9.81$ m/s² et suivant le modèle des déformation planes. Elle est présentée à la figure 2.

L'erreur de déplacement v pour le cas statique ou Φ pour l'analyse modale est mesurée le long de la fibre neutre. On constate que le modèle numérique 2D ne converge pas exactement vers la solution analytique 1D.



4.2 Cylindre creux

Un cylindre creux de rayon interne r_1 , de rayon externe r_2 et de hauteur $H \gg R$ est soumis à une pression uniforme p_1 sur sa face interne et à une pression uniforme p_2 sur sa face externe.

La solution a ce problème peut s'obtenir

- analytiquement,
- avec une simulation en déformation plane ou bien
- avec une simulation en axisymétrie.

4.2.1 Solution analytique

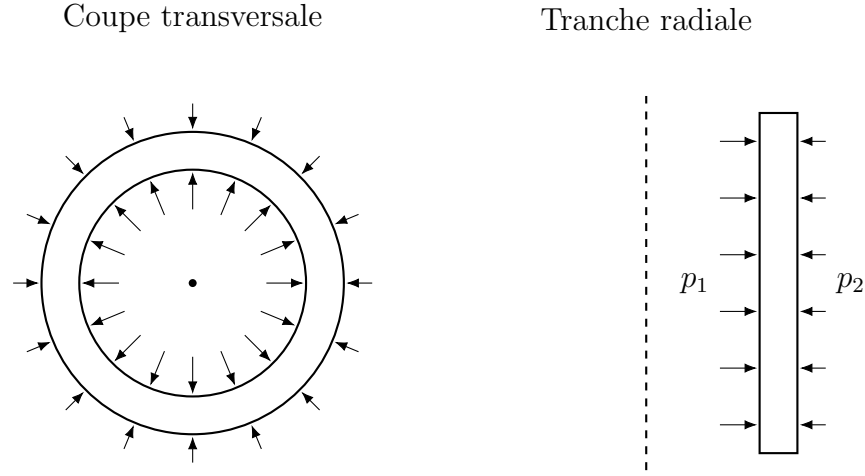


Figure 3. Cylindre creux soumis à une pression uniforme sur ses faces.

On peut faire l'hypothèse que le déplacement est radial et indépendant des coordonnées axiales et angulaires : $\mathbf{u}(r, \theta, z) = u(r) \mathbf{e}_r$. On peut en dériver $\boldsymbol{\epsilon}$ et $\boldsymbol{\sigma}$ grâce aux équations (2) et (3) :

$$\sigma_{rr} = (2\mu + \lambda)\partial_r u + \lambda \frac{u}{r} \quad (88)$$

$$\sigma_{\theta\theta} = \lambda\partial_r u + (2\mu + \lambda)\frac{u}{r} \quad (89)$$

La conservation de la quantité de mouvement dans la direction radiale donne alors

$$\partial_r^2 u + \frac{1}{r}\partial_r u - \frac{u}{r^2} = 0 \quad (90)$$

Grâce au changement de variable $t = \ln(r)$, on obtient la solution à cette équation différentielle

$$\partial_t^2 u = u \quad \Longleftrightarrow \quad u(t) = C_1 e^t + C_2 e^{-t} \quad \Longleftrightarrow \quad u(r) = Ar + \frac{B}{r}. \quad (91)$$

Les conditions frontières en r_1 et r_2 nous permettent de déterminer les constantes A et B :

$$-p_i = \sigma_{rr}(r_i) = 2(\mu + \lambda)A - 2\mu B \frac{1}{r_i^2} = \frac{E}{1 + \nu} \left[\frac{A}{1 - 2\nu} - \frac{B}{r_i^2} \right] \quad i = 1, 2 \quad (92)$$

$$\Rightarrow \begin{cases} A = \frac{(1 + \nu)(1 - 2\nu)}{E} \frac{p_1 r_1^2 - p_2 r_2^2}{r_2^2 - r_1^2} \\ B = -\frac{1 + \nu}{E} \frac{r_1^2 r_2^2}{r_2^2 - r_1^2} (p_2 - p_1). \end{cases} \quad (93)$$

Pour résumer, avec $r_1 = \alpha r_2$ et $\beta p_1 = p_2$, on a donc

$$\frac{u(r)}{r_2} = \frac{p_1}{E} \frac{1 + \nu}{1 - \alpha^2} \left[(1 - 2\nu)(\alpha^2 - \beta) \frac{r}{r_2} + \alpha^2(1 - \beta) \frac{r_2}{r} \right] \quad (94)$$

$$\frac{\sigma_{rr}(r)}{p_1} = \frac{1}{1 - \alpha^2} \left[(\alpha^2 - \beta) - \alpha^2(1 - \beta) \left(\frac{r_2}{r} \right)^2 \right] \quad (95)$$

$$\frac{\sigma_{\theta\theta}(r)}{p_1} = \frac{1}{1 - \alpha^2} \left[(\alpha^2 - \beta) + \alpha^2(1 - \beta) \left(\frac{r_2}{r} \right)^2 \right] \quad (96)$$

$$\frac{\sigma_{zz}(r)}{p_1} = 2\nu \frac{\alpha^2 - \beta}{1 - \alpha^2} \quad (97)$$

4.3 Solutions numériques

L'analyse de convergence du code a été réalisée sur un cylindre creux de rayon interne $r_1 = 10$ cm, de rayon externe $r_2 = 15$ cm, de hauteur $H = 20$ cm, avec $E = 40$ GPa, $\rho = 2300$ kg/m³ et $\nu = 0.2$. Elle est présentée à la figure 4.

Dans le cas plane, le vecteur déplacement $\mathbf{u}$ et le tenseur des contraintes $\boldsymbol{\sigma}$ sont transformées des coordonnées cartésiennes en coordonnées cylindriques. L'erreur L_2 est ensuite calculée sur base de la solution analytique :

$$e^2(\mathbf{u}^h) = \frac{\int_{\Omega} \|\mathbf{u}^h - \mathbf{u}^*\|^2 \, d\mathbf{x}}{\int_{\Omega} \|\mathbf{u}^*\|^2 \, d\mathbf{x}} = \frac{\int_{\Omega} (u_r^h - u_r^*)^2 \, d\mathbf{x}}{\int_{\Omega} (u_r^*)^2 \, d\mathbf{x}} \quad (98)$$

$$e^2(\sigma_i^h) = \frac{\int_{\Omega} (\sigma_i^h - \sigma_i^*)^2 \, d\mathbf{x}}{\int_{\Omega} (\sigma_i^*)^2 \, d\mathbf{x}} \quad \text{avec } i \in \{rr, \theta\theta, zz\} \quad (99)$$

$$(100)$$

On constate que les modèles 2D et 3D convergent à la même vitesse vers la solution, mais que le modèle 3D présente tout de même une erreur légèrement plus faible pour une taille de maille similaire.

On constate également que le taux de convergence pour les contraintes est superlinéaire quel que soit la méthode de calcul utilisée (moindres carrés ou moyenne pondérée, cf. section 2.6 pour plus de détails).

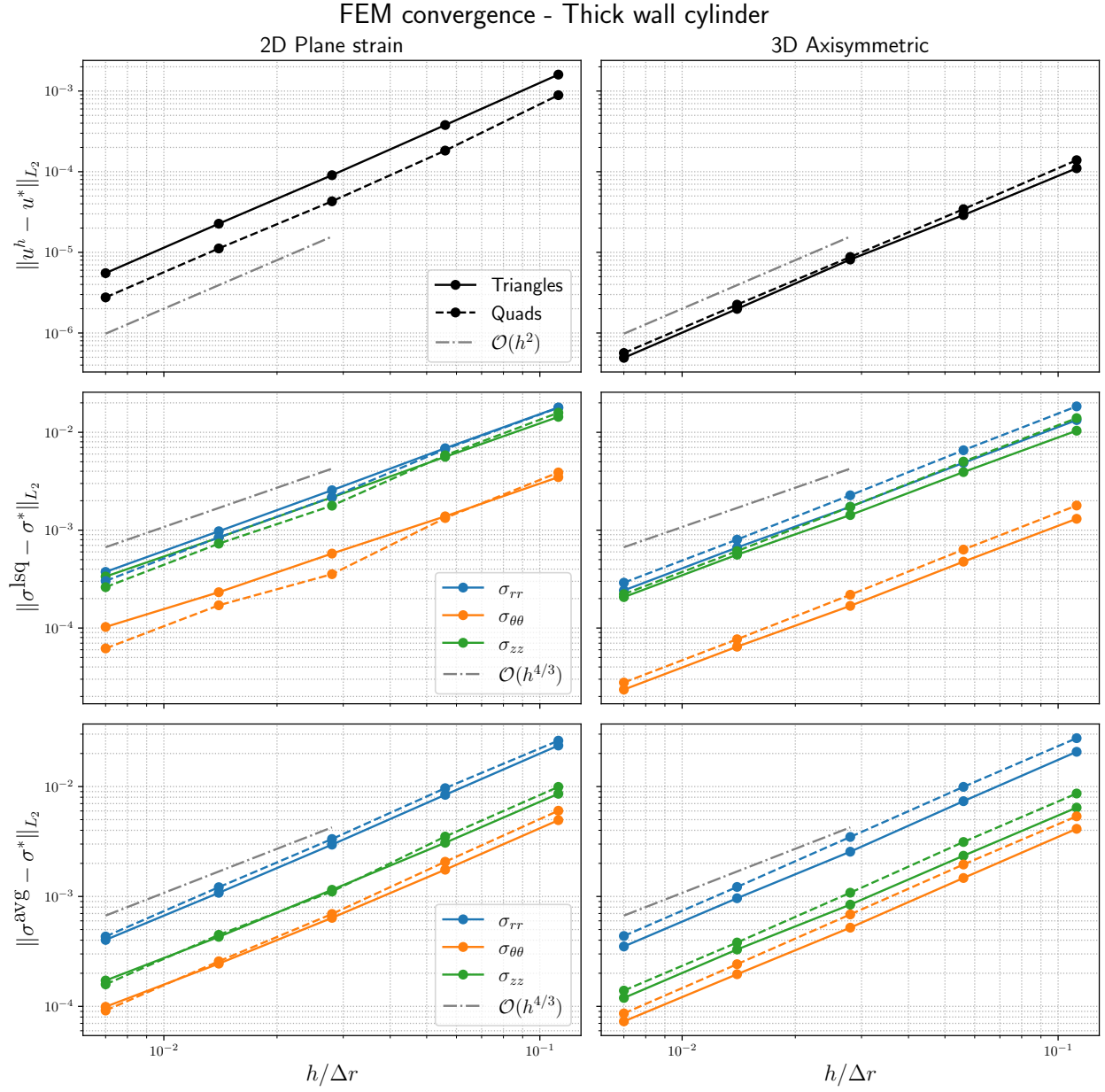


Figure 4. Analyse de convergence pour un cylindre creux.